# Diabetes in Pima Indians

Katherine Rodriguez

2026-07-10

## Contents

**1 Summary** **2**

**2 Load and prepare data** **2**

2.1 Load libraries . . . . . 2

2.2 Load the data . . . . . 2

2.3 Lets take a look at the data . . . . . 2

**3 Tidy the data** **3**

3.1 Replace values of zeros with NAs for BMI, glucose, and blood pressure recode outcome . . . . 3

**4 Train and test split** **3**

**5 Recipe: impute, dummy, and normalize** **3**

**6 Model Building** **3**

6.1 Logistic Regression Model . . . . . 3

6.2 Random Forest Model . . . . . 4

**7 Predictions on test set** **4**

**8 Metrics: Accuracy and ROC AUC** **4**

**9 ROC Curve and Variable Importance for Random Forest** **5**

**10 Visualizations** **6**

# 1 Summary

This script uses the Pima Indians diabetes dataset to build predictive models for diabetes status. After cleaning impossible values (e.g., 0 for glucose, blood pressure, BMI) and recoding the outcome, we fit a logistic regression and a random forest classifier using tidymodels. We compare performance on a train/test split using accuracy and ROC AUC, visualize the ROC curve, and examine variable importance to understand which health metrics are most strongly associated with diabetes.

# 2 Load and prepare data

## 2.1 Load libraries

```
library(tidyverse)
library(tidymodels)
```

## 2.2 Load the data

```
diabetes <- read_csv("data/diabetes.csv")
```

```
## Rows: 500 Columns: 7
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): pregnancies
## dbl (6): glucose, blood_pressure, insulin, bmi, age, outcome
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

## 2.3 Lets take a look at the data

```
head(diabetes)
```

```
## # A tibble: 6 x 7
##   pregnancies glucose blood_pressure insulin   bmi   age outcome
##   <chr>         <dbl>          <dbl>   <dbl> <dbl> <dbl>   <dbl>
## 1 1-2              99             60     160  36.6    21       0
## 2 3+              136             70       0  31.2    22       1
## 3 3+              158             70       0  29.8    63       0
## 4 None            123             88       0  35.2    29       0
## 5 3+              168             88     321  38.2    40       1
## 6 3+              134             80     370  46.2    46       1
```

# 3 Tidy the data

## 3.1 Replace values of zeros with NAs for BMI, glucose, and blood pressure recode outcome

```
diabetes <- diabetes |>
  mutate_at(c('bmi', 'glucose', 'blood_pressure'), ~na_if(., 0))|>
  mutate_if(is.character, as.factor) |>
  mutate(
    outcome = recode_factor(outcome, "0" = "No diabetes", "1" = "Diabetes")
  )
```

# 4 Train and test split

```
set.seed(630)
split_diab <- initial_split(diabetes, prop = 0.75, strata = outcome)
train_diab <- training(split_diab)
test_diab <- testing(split_diab)
```

# 5 Recipe: impute, dummy, and normalize

```
rec_diab <- recipe(outcome ~ pregnancies + glucose + blood_pressure + bmi + age + insulin,
                   data = train_diab) |>
  step_impute_median(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors()) |>
  step_normalize(all_numeric_predictors())
```

# 6 Model Building

## 6.1 Logistic Regression Model

```
logit_mod <- logistic_reg(mode = "classification") |>
  set_engine("glm")

logit_wf <- workflow() |>
  add_recipe(rec_diab) |>
  add_model(logit_mod)

logit_fit <- logit_wf |>
  fit(data = train_diab)
```

### 6.2 Random Forest Model

```
rf_mod <- rand_forest(mode = "classification", trees = 500) |>
  set_engine("ranger", importance = "permutation")

rf_wf <- workflow() |>
  add_recipe(rec_diab) |>
  add_model(rf_mod)

rf_fit <- rf_wf |>
  fit(data = train_diab)
```

## 7 Predictions on test set

```
logit_pred <- predict(logit_fit, new_data = test_diab, type = "prob")|>
  bind_cols(predict(logit_fit, new_data = test_diab, type = "class")) |>
  bind_cols(test_diab |> select(outcome)) |>
  rename(.pred_class = .pred_class)

rf_pred <- predict(rf_fit, new_data = test_diab, type = "prob") |>
  bind_cols(predict(rf_fit, new_data = test_diab, type = "class")) |>
  bind_cols(test_diab |> select(outcome)) |>
  rename(.pred_class = .pred_class)
```

## 8 Metrics: Accuracy and ROC AUC

```
metrics_logit <- tibble(
  model = "Logistic regression",
  accuracy = accuracy(logit_pred, truth = outcome, estimate = .pred_class)$.estimate,
  roc_auc = roc_auc(logit_pred, truth = outcome, .pred_Diabetes, event_level = "second")$.estimate

)

metrics_rf <- tibble(
  model = "Random forest",
  accuracy = accuracy(rf_pred, truth = outcome, estimate = .pred_class)$.estimate,
  roc_auc = roc_auc(rf_pred, truth = outcome, .pred_Diabetes, event_level = "second")$.estimate

)

bind_rows(metrics_logit, metrics_rf)
```

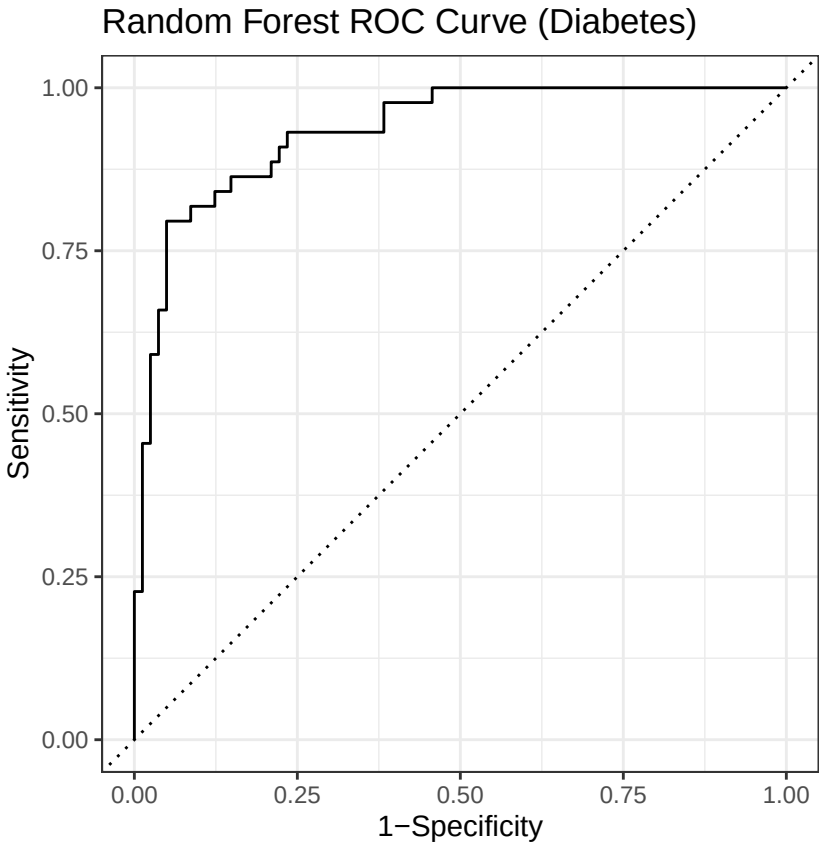
```
## # A tibble: 2 x 3
##   model               accuracy roc_auc
##   <chr>                  <dbl>   <dbl>
## 1 Logistic regression    0.864   0.908
## 2 Random forest          0.896   0.934
```

**Discussion**

Both models perform well, with an accuracy of 0.864 and an ROC AUC of 0.908 for logistic regression, and the random forest model yielded an accuracy of 0.896, and an ROC AUC of 0.934.

# 9 ROC Curve and Variable Importance for Random Forest

```
roc_plot <- rf_pred |>
roc_curve(truth = outcome, .pred_Diabetes, event_level = "second") |>
  autoplot() +
  labs(
    title = "Random Forest ROC Curve (Diabetes)",
    x = "1-Specificity",
    y = "Sensitivity"
  )
roc_plot
```




```
#Save ROC curve
ggsave("results/roc_curve.png", roc_plot, width = 6, height = 4)

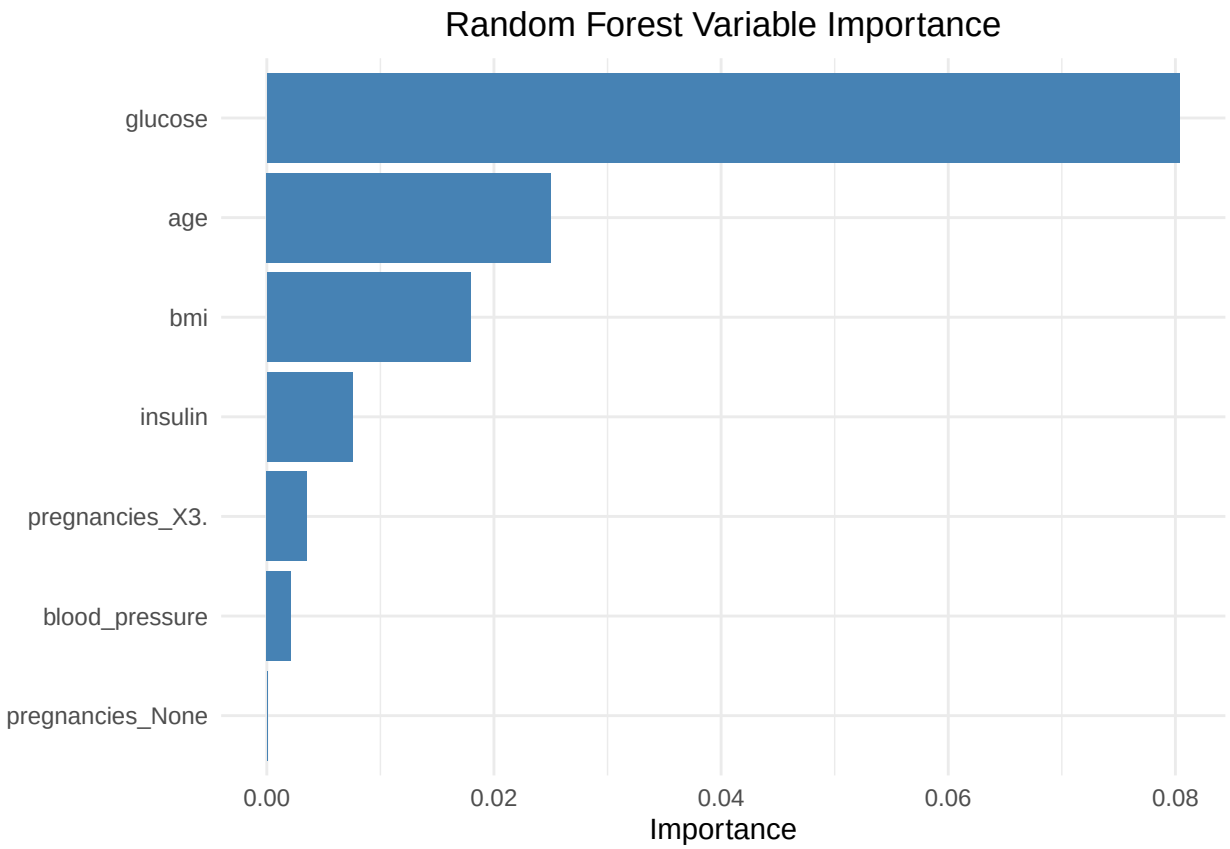
#Variable importance (random forest)
rf_fitb <- extract_fit_parsnip(rf_fit)$fit
imp <- tibble(
```

```
  variable = names(rf_fitb$variable.importance),
  importance = as.numeric(rf_fitb$variable.importance)
) |>
  arrange(desc(importance))
```

## 10 Visualizations

```
varimp_plot <- ggplot(imp, aes(x = reorder(variable, importance), y = importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Random Forest Variable Importance",
    x = NULL,
    y = "Importance"
  ) +
  theme_minimal()+
  theme(
    plot.title = element_text(hjust = 0.5))

varimp_plot
```



```r
#Save variable importance chart
ggsave("results/variable_importance.png", varimp_plot, width = 6, height = 4)
```

### Discussion

As the metrics revealed, both the logistic regression and the random forest models performed well. Nonetheless, the random forest outperformed logistic regression on both metrics, slightly higher accuracy (0.896 vs. 0.864) and a stronger ROC AUC (0.934 vs. 0.908) The variable importance chart reveals glucose is the strongest predictor of diabetes, consistent with clinical explanations of diabetes and what is likely to cause a higher diabetes risk.